Rumbo Sur – Diagrama de clases final (SOLID y GRASP)

Excepciones

- «Exception» ReglaNegocioException
- «Exception» PagoRequeridoException
- «Exception» RequisitosNoCumplidosException
- «Exception» CuposInsuficientesException
- «Exception» EntidadNoEncontradaException
- «Exception» OperacionNoPermitidaException
- «Exception» ReservaNoCancelableException

Modelo del negocio

- Comparable
- *Persona* — #nombre: String; #apellido: String; #documento: String; #telefono: String; +obtenerNombreCompleto(): String
- *Empleado* — #legajo: int; *+getRol(): String*
- Administrador — +getRol(): String
- Coordinador — +getRol(): String
- Guia — -idiomas: List<String>; -categoriasHabilitadas: List<CategoriaTour>; -honorarioPorDia: double; +getRol(): String; +estaHabilitadoPara(cat: CategoriaTour): boolean; +estaDisponible(inicio: LocalDate, fin: LocalDate): boolean; +calcularHonorarios(salida: Salida): double
- Pasajero — -fechaNacimiento: LocalDate; -tipo: TipoPasajero; -contactoEmergencia: String; -restricciones: String; +calcularEdad(fecha: LocalDate): int; +tipoEsCompatible(fecha: LocalDate): boolean; +obtenerFactorTarifa(fecha: LocalDate): double
- Vendedor — -comision: double; +getRol(): String
- Salida — -id: long; -fechaInicio: LocalDate; -fechaFin: LocalDate; -cupoMaximo: int; -estado: EstadoSalida; -listaEmitida: boolean; +cuposOcupados(hoy: LocalDate): int; +cuposDisponibles(hoy: LocalDate): int; +verificarVencimientos(hoy: LocalDate): void; +puedeParticipar(datos: DatosPasajero): boolean; +obtenerTarifa(): Tarifa; +diasHastaInicio(hoy: LocalDate): int; +exigePagoTotal(hoy: LocalDate): boolean; +totalServiciosAdicionales(): double; +agregarReserva(reserva: Reserva): void; +asignarGuia(guia: Guia): void; +asignarVehiculo(vehiculo: Vehiculo): void; +emitirLista(): List<Pasajero>; +actualizarEstado(hoy: LocalDate): void; +validarCondicionesInicio(): void; +iniciar(): void; +finalizar(): void; +compareTo(otra: Salida): int
- Reserva — -numero: long; -fecha: LocalDate; -estado: EstadoReserva; -total: double; -vencimientoPago: LocalDate; +agregarPasajero(datos: DatosPasajero): Pasajero; +incluirServiciosObligatorios(): void; +agregarServicioAdicional(p: Pasajero, s: ServicioAdicional): void; +totalServiciosAdicionales(): double; +registrarTotal(total: double): void; +iniciarRetencion(hoy: LocalDate): void; +estaVencida(hoy: LocalDate): boolean; +ocupaCupos(hoy: LocalDate): boolean; +cantidadPasajeros(): int; +registrarPago(importe: double, recargo: double, medio: MedioPago, cuotas: int, fecha: LocalDate): Pago; +totalAbonado(): double; +calcularSaldo(): double; +seniaCubierta(): boolean; +medioDePagoUsado(): MedioPago; +obtenerSaldo(): Salida; +puedeCancelarse(): boolean; +confirmar(): void; +finalizar(): void; +anular(cancelacion: Cancelacion): void; +anularPorVencimiento(): void; +reemplazarPasajero(documento: String, datos: DatosPasajero, hoy: LocalDate): void; +registrarAjuste(motivo: String, importe: double, fecha: LocalDate): NotaAjuste
- Liquidacion — -fecha: LocalDate; -dias: int; -monto: double
- Pago — -fecha: LocalDate; -importe: double; -recargo: double; -medio: MedioPago; -cuotas: int
- NotaAjuste — -fecha: LocalDate; -motivo: String; -importe: double
- Cliente — -correo: String; -paisResidencia: String
- ServicioAdicional — -nombre: String; -precio: double; -tipo: TipoServicio; -obligatorio: boolean
- Cancelacion — -fecha: LocalDate; -motivo: String; -porcentajeDevolucion: int; -importeDevuelto: double; -medioDevolucion: MedioPago
- Actividad — -dia: int; -orden: int; -horario: LocalTime; -descripcion: String; -lugar: String
- Proveedor — -nombre: String; -cuit: String; -contacto: String; -tipoServicio: TipoProveedor
- Vehiculo — -patente: String; -capacidad: int; -tipo: TipoVehiculo
- *Tour* — -codigo: String; -nombre: String; -descripcion: String; -duracionDias: int; -cupoMinimo: int; -cupoMaximo: int; *+getCategoria(): CategoriaTour*; +requiereSeguro(): boolean; +puedeParticipar(datos: DatosPasajero, fecha: LocalDate): boolean; +obtenerTarifa(fecha: LocalDate): Tarifa; +obtenerServiciosObligatorios(): List<ServicioAdicional>
- TourAventura — -nivelDificultad: NivelDificultad; -edadMinima: int; +getCategoria(): CategoriaTour; +requiereSeguro(): boolean; +puedeParticipar(datos: DatosPasajero, fecha: LocalDate): boolean
- TourCultural — +getCategoria(): CategoriaTour
- TourNaturaleza — +getCategoria(): CategoriaTour
- Tarifa — -temporada: Temporada; -vigenciaDesde: LocalDate; -vigenciaHasta: LocalDate; -precioBase: double; +estaVigente(fecha: LocalDate): boolean

Notas:
- cuposOcupados cuenta los pasajeros de las reservas pendientes vigentes y confirmadas; las anuladas no ocupan lugares. Los menores de 3 años también ocupan un lugar aunque no abonen tarifa.
- verificarVencimientos anula las reservas vencidas.
- finalizar() pasa la salida a FINALIZADA y finaliza sus reservas vigentes.
- importe es el capital abonado (sin recargo) y se usa para saldo, seña y devolucion. recargo solo se informa con tarjeta en cuotas (7 %) y no cambia el total.

Control e interfaz

- DatosPasajero — -nombre: String; -apellido: String; -documento: String; -fechaNacimiento: LocalDate; -tipo: TipoPasajero; -contactoEmergencia: String; -restricciones: String
- ControladorOperaciones — +registrarTour(tour: Tour): void; +registrarTarifa(codigoTour: String, tarifa: Tarifa): void; +registrarProveedor(proveedor: Proveedor): void; +registrarGuia(guia: Guia): void; +registrarVehiculo(vehiculo: Vehiculo): void; +programarSalida(codigoTour: String, inicio: LocalDate, fin: LocalDate, cupo: int): Salida; +actualizarEstadoSalida(idSalida: long, hoy: LocalDate): void; +asignarRecursos(idSalida: long, legajoGuia: int, patente: String): void; +emitirListaPasajeros(idSalida: long): List<Pasajero>; +iniciarSalida(idSalida: long): void; +finalizarSalida(idSalida: long): void; +liquidarHonorarios(idSalida: long, hoy: LocalDate): Liquidacion
- MenuConsola — +mostrarMenu(): void
- GeneradorReportes — +reporteOcupacionPorSalida(): String; +reporteVentasPorVendedor(): String; +reporteIngresosPorTour(): String; +reporteCancelaciones(): String
- ControladorReservas — +registrarCliente(nombre: String, apellido: String, documento: String, telefono: String, correo: String, pais: String): Cliente; +iniciarReserva(documentoCliente: String, idSalida: long, legajoVendedor: int, datos: List<DatosPasajero>, hoy: LocalDate): Reserva; +agregarServicioAdicional(reserva: Reserva, p: Pasajero, s: ServicioAdicional): void; +finalizarReserva(reserva: Reserva, medio: MedioPago, cuotas: int, hoy: LocalDate): Reserva; +registrarPago(numero: long, importe: double, medio: MedioPago, cuotas: int, hoy: LocalDate): Pago; +verificarVencimientos(idSalida: long, hoy: LocalDate): void; +cancelarReserva(numero: long, motivo: String, porAgencia: boolean, hoy: LocalDate): Cancelacion; +reemplazarPasajero(numero: long, documento: String, datos: DatosPasajero, hoy: LocalDate): void; +registrarAjuste(numero: long, motivo: String, importe: double, hoy: LocalDate): NotaAjuste

Raiz y persistencia

- Agencia — -tours: List<Tour>; -salidas: List<Salida>; -clientes: List<Cliente>; -reservas: List<Reserva>; -empleados: List<Empleado>; -vehiculos: List<Vehiculo>; -proveedores: List<Proveedor>; +buscarTour(codigo: String): Tour; +buscarSalida(id: long): Salida; +buscarCliente(documento: String): Cliente; +buscarReserva(numero: long): Reserva; +buscarEmpleado(legajo: int): Empleado; +buscarVehiculo(patente: String): Vehiculo; +siguienteNumeroReserva(): long; +agregarReserva(reserva: Reserva): void
- *RepositorioAgencia* — +guardar(agencia: Agencia): void; +cargar(): Agencia
- RepositorioAgenciaArchivo — -rutaArchivo: String; +guardar(agencia: Agencia): void; +cargar(): Agencia

Nota: Agencia es la raiz del sistema: agrupa las colecciones de Tour, Salida, Reserva, Cliente, Empleado, Vehiculo y Proveedor.

Servicios de calculo, pagos y cancelaciones

- GestorCancelaciones — +cancelar(reserva: Reserva, motivo: String, porAgencia: boolean, hoy: LocalDate): Cancelacion
- GestorPagos — +registrarPago(reserva: Reserva, importe: double, medio: MedioPago, cuotas: int, fecha: LocalDate): Pago
- CalculadoraPrecioReserva — -promociones: List<Promocion>; +CalculadoraPrecioReserva(promociones: List<Promocion>); +calcularTotal(reserva: Reserva): double
- *Promocion* — +calcularDescuento(reserva: Reserva, subtotalTour: double): double
- PromocionAnticipada — +calcularDescuento(reserva: Reserva, subtotalTour: double): double
- PromocionGrupo — +calcularDescuento(reserva: Reserva, subtotalTour: double): double

Enumeraciones

- EstadoSalida: PROGRAMADA, CONFIRMADA, COMPLETA, EN_CURSO, FINALIZADA, CANCELADA
- NivelDificultad: BAJO, MEDIO, ALTO
- Temporada: BAJA, MEDIA, ALTA
- CategoriaTour: AVENTURA, CULTURAL, NATURALEZA
- TipoVehiculo: MICRO, COMBI, CAMIONETA
- MedioPago: EFECTIVO, TRANSFERENCIA, TARJETA
- TipoPasajero: ADULTO, MENOR, JUBILADO
- EstadoReserva: PENDIENTE, CONFIRMADA, ANULADA, FINALIZADA
- TipoProveedor: HOTEL, RESTAURANTE, TRANSPORTISTA
- TipoServicio: EXCURSION, TRASLADO, SEGURO

Azul: clase nueva por SOLID/GRASP
Amarillo: clase modificada respecto del diagrama inicial
Sin color: sin cambios